Proof-of-Concept Kit — Microsoft Defender for Cloud

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 6

Effort . . . 6

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Cloud 6

What Microsoft Defender for Cloud is . . . 6

What a Proof-of-Concept Kit is . . . 6

Why this matters . . . 7

How it works . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

Connections and authentication used . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud 8

2.1 A pilot Azure subscription (or resource group) . . . 8

2.2 Azure roles to enable and tear down . . . 9

2.3 Awareness of Defender plan pricing . . . 9

2.4 A pilot workload to protect (optional VM/resource) . . . 9

2.5 A Log Analytics workspace (where required) . . . 9

2.6 Evaluation criteria agreed up front . . . 9

2.7 Read access for validation . . . 9

Checklist . . . 10

Global Requirements — Defender (category-level) 10

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 10

G4. Onboarding reach to endpoints / identity / workloads . . . 11

G5. Network access to Defender service endpoints . . . 11

G6. A pilot ring . . . 11

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud 11

Naming (set once, reuse verbatim) . . . 12

Step 1 — Assess readiness (read-only) . . . 12
Step 2 — Enable Defender for Cloud on the pilot subscription . . . 12
Step 3 — Generate benign test signal . . . 12
Step 4 — Validate and demonstrate the detections . . . 13
Step 5 — Score against your criteria . . . 13
Step 6 — Tear down (clean rollback + stop billing) . . . 13
Step 7 — Decide: expand or stop . . . 13
5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud 14
Test 1 — Defender plan enabled on the pilot subscription . . . 14
Test 2 — Benign signal generated . . . 14
Test 3 — Recommendation raised (CSPM) . . . 14
Test 4 — Secure Score reflects posture . . . 14
Test 5 — Security alert fired . . . 14
Test 6 — Regulatory-compliance data populated . . . 15
Test 7 — Scorecard produced . . . 15
Test 8 — Clean teardown (and billing stopped) . . . 15
6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud 15
Plan not enabling . . . 15
Signal not generated . . . 15
No recommendation . . . 15
No Secure Score . . . 16
No security alert . . . 16
No compliance data . . . 16
Script cannot authenticate . . . 16
Teardown left residue . . . 16
7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Cloud 16
Keep the PoC scoped and reversible . . . 16
Mind the cost — disable at teardown . . . 17
Benign signal only — never real attacks . . . 17
Use a disposable test resource for CSPM . . . 17
Set the bar before you run . . . 17
Allow warm-up before judging . . . 17
Tear down completely and confirm . . . 17
Defer the decisions to the owners . . . 17
Where to go next . . . 18
8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud 18

Documentation 18
PowerShell scripts (Az PowerShell / Az.Security + Microsoft Graph + KQL) 18
What the buyer receives 19
Outcome 19
Appendix A — PowerShell Scripts 20

Executive Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud

White-label · Defender

Deciding on Microsoft Defender for Cloud shouldn't rest on a slide deck. This Proof-of-Concept (PoC) Kit stands up a **scoped, safe** proof of concept in your own environment: it enables Defender for Cloud on a single **pilot subscription** (or resource group), generates **benign, simulated** test signal (Defender for Cloud's own **sample security alerts**, a deliberately-misconfigured **benign test resource** that produces a posture recommendation, and the **EICAR** test file on a pilot VM — never a real attack, exploit, or malware), demonstrates the headline **cloud security posture management (CSPM) & workload protection (CWPP)** capabilities on that signal, scores the result against your evaluation criteria, and then **cleanly tears the PoC back down**.

The kit is written for an administrator with no prior Defender experience. Everything is **pilot-scoped and reversible**: you enable Defender on one subscription, prove the value, capture a scorecard, and disable it — with no other subscription touched and no production cost surprises.

What you get

- A plain-language **Solution Overview** of Defender for Cloud (CSPM + CWPP) and how a PoC proves it on real-but-safe signal.
- A **Requirements Guide** of the PoC-specific prerequisites (the category-wide ones live in `global-requirements-defender.md`).
- An **Implementation Guide** — exact, ordered steps to enable Defender on a pilot subscription, generate benign signal, demonstrate the detections, score, and tear down.
- Five **PowerShell scripts** (Az PowerShell / Az.Security + Microsoft Graph) with **KQL** queries: a `-WhatIf`-gated pilot enable, a **benign** test-signal helper, a demo/validate check, a criteria scorecard, and a `-WhatIf`-gated teardown.
- **Testing**, **Troubleshooting**, and **Best Practices** guides.

What this delivers

A concrete, **in-environment proof** that Microsoft Defender for Cloud delivers CSPM & workload protection — a **Secure Score** and **recommendations** from a benign misconfiguration, a **security alert** from a sample alert and an EICAR trigger on a pilot VM, the **regulatory-compliance** dashboard populated, the events visible in the portal and hunting, and a **scorecard** mapping each observed capability to your pass/fail criteria — all torn down cleanly at the end.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — here scoped as a PoC: assess readiness, enable Defender on a pilot subscription, generate benign signal, validate and score, then either expand or tear down.

Boundary

This is an **evaluation kit**, not a production rollout. It proves Defender for Cloud on one subscription and then disables it. Multi-subscription/multi-cloud onboarding, CSPM remediation at scale, and long-term operations are separate adjacent solutions (see *Where to go next*). All test signal is **benign and simulated**. Enabling a Defender plan can incur **cost while active — verify current pricing and disable at teardown**. Remediation and regulatory determinations remain the customer's security/compliance team's.

Effort

A competent administrator can complete the full PoC — enable, signal, validate, score, and teardown — in a short, focused engagement on one pilot subscription. Defender plans are billed while enabled, so keep the PoC short and disable at teardown. Every step is reversible.

1. Solution Overview — Proof-of-Concept Kit: Microsoft Defender for Cloud

What Microsoft Defender for Cloud is

Microsoft Defender for Cloud is Microsoft's **cloud security posture management (CSPM)** and **cloud workload protection platform (CWPP)** for Azure, AWS, GCP, and hybrid resources. It is managed from the Azure portal (`portal.azure.com` → **Microsoft Defender for Cloud) and surfaces alerts into the unified Defender portal at` security.microsoft.com`. Its headline capabilities are:

- **Secure Score & recommendations (CSPM)** — continuous assessment of resource configuration against security best practice, with a single posture score to trend.
- **Regulatory compliance** — maps your posture to standards (e.g. Microsoft Cloud Security Benchmark and others). *Specific standards/controls may be plan-gated — verify current availability.*
- **Defender workload plans (CWPP)** — threat protection for specific resource types (Defender for Servers, Storage, SQL, Containers/AKS, Key Vault, App Service, and more). *Each plan is separately priced — verify current pricing.*
- **Security alerts** — runtime threat detections for the enabled workloads, including MDE-backed detection on servers.
- **Advanced hunting / continuous export** to a Log Analytics workspace or Sentinel.

What a Proof-of-Concept Kit is

A PoC Kit is a **customer-runnable evaluation**, not a production rollout and not a sales demo. Its job is to answer one question — *does Defender for Cloud actually surface our cloud misconfigurations and threats?* — with evidence, safely, in your own environment. It does that by:

1. **Enabling** Defender for Cloud on a single **pilot subscription** (or resource group) — CSPM plus one workload plan.
2. **Generating benign test signal** — Defender for Cloud's built-in **sample security alerts**, a deliberately-misconfigured **benign test resource** (e.g. a storage account with public access) that produces a posture recommendation, and the **EICAR** test file on a pilot VM to raise a

Defender-for-Servers alert. **No real attack, exploit, or malware is ever used.**

3. **Demonstrating** that the signal produces the expected outcomes — a recommendation and Secure Score change, a security alert, and regulatory-compliance visibility.

4. **Scoring** what you observed against your evaluation criteria.

5. **Tearing down** — deleting the test resource, disabling the Defender plan, and resolving the sample alerts.

Why this matters

Cloud misconfigurations are the leading cause of cloud breaches, and a vendor deck can't show you what Defender for Cloud surfaces about *your* subscription. A scoped PoC replaces "trust the deck" with "watch it flag the misconfig and the threat." Because it runs on **one pilot subscription** with a **benign test resource** and is **disabled at teardown**, you get proof without a broad rollout or an open-ended bill.

How it works

1. **Assess** readiness with `POCMDC-Enable.ps1 -WhatIf` — Azure roles, the pilot subscription, and current Defender-for-Cloud state.

2. **Configure & enable** — turn on **CSPM** and one **Defender workload plan** on the pilot subscription, and create the pilot resource group `MDC PoC – Pilot RG – Pilot`.

3. **Generate benign signal** with `POCMDC-TestSignal.ps1` — trigger sample security alerts, deploy the benign misconfigured test resource, and (optionally) drop EICAR on a pilot VM.

4. **Validate & demonstrate** with `POCMDC-Validate.ps1` — confirm the recommendation, Secure Score, security alerts, and regulatory-compliance data populated.

5. **Score** with `POCMDC-Scorecard.ps1` against your criteria.

6. **Tear down** with `POCMDC-Teardown.ps1 -WhatIf` first, then for real — delete the test resource, disable the Defender plan, resolve sample alerts.

What you can control

• **Pilot subscription/RG** — exactly one subscription (or resource group); no other subscription is touched.

• **Which capabilities to prove** — Secure Score/recommendations, security alerts, regulatory compliance, workload protection.

• **Which Defender plan** — enable one workload plan for the PoC (kept short to control cost).

• **Scorecard criteria** — the pass/fail bar you evaluate against.

• **Teardown** — full, reversible removal, including disabling the plan to stop billing.

Limitations (be honest with buyers)

• **Defender workload plans are billed while enabled** — the PoC keeps the window short and disables at teardown. *Verify current pricing before enabling.*

• Some standards/controls and CSPM features are **plan-gated** (e.g. Defender CSPM vs foundational CSPM). *Verify current availability in your tenant.*

• Test signal is **benign and simulated only** — sample alerts, a benign misconfiguration, and EICAR. This proves the posture/detection pipeline, not resilience to a real cloud attack.

• **Secure Score and recommendations populate over time** — allow warm-up after enabling before judging.

• This is a **PoC**, not a production rollout — it stays on one subscription and is designed to be disabled. Multi-cloud onboarding is a separate solution.

• Defender for Cloud improves posture and detection; it does not replace remediation, IaC hardening, or the security team's decisions.

Connections and authentication used

• The **Azure portal** (`portal.azure.com` → **Microsoft Defender for Cloud**) for plans, Secure Score, recommendations, alerts, and regulatory compliance.

• **Az PowerShell** (`Az.Security`, `Az.Resources`, `Az.Storage`) for read-only discovery, enable/teardown, and validation.

• **Microsoft Graph Security** / the unified Defender portal where Defender-for-Cloud alerts correlate into incidents.

• Least-privilege roles: **Security Admin** / **Contributor** (scoped to the pilot subscription) for enable/teardown, **Security Reader** for read-only work, and **Owner** only where a plan requires it.

The standard deployment model

> ***Assess readiness → configure/enable CSPM + one workload plan on a pilot subscription → generate benign posture + threat signal → validate the detections and score → expand or tear down.*** *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, run here as a safe, reversible, cost-bounded PoC.*

2. Requirements Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud

Verify **every** requirement before enabling anything. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running a scoped Defender-for-Cloud proof of concept**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 A pilot Azure subscription (or resource group)

Description: One **non-production Azure subscription** (or a scoped resource group) to run the PoC.
Why: Defender for Cloud is enabled per subscription; the PoC deliberately uses one so it can be disabled

cleanly with no impact elsewhere. **Verify:** `POCMDC-Enable.ps1 -WhatIf` lists the subscription and its current Defender state; you can sign in with `Connect-AzAccount`. **Implement-if-missing:** Choose a test/dev subscription (or create a pilot resource group `MDC PoC – Pilot RG – Pilot`).

2.2 Azure roles to enable and tear down

Description: Security Admin and **Contributor** (or Owner where a plan requires it) on the pilot subscription. **Why:** Enabling/disabling Defender plans and deploying/removing the benign test resource require these roles. **Verify:** `portal.azure.com` → **Subscriptions** → **Access control (IAM)** confirms your roles; `POCMDC-Enable.ps1 -WhatIf` reports permission gaps. **Implement-if-missing:** Assign Security Admin + Contributor on the pilot subscription for the running account.

2.3 Awareness of Defender plan pricing

Description: Acknowledgement that **Defender workload plans are billed while enabled**. **Why:** A PoC that leaves a plan on becomes an open-ended cost; the kit keeps the window short and disables at teardown. **Verify:** Review current Defender-for-Cloud pricing for the plan you will enable. *Verify current pricing before enabling.* **Implement-if-missing:** Confirm the cost owner accepts the short PoC billing window, and diarise the teardown.

2.4 A pilot workload to protect (optional VM/resource)

Description: Optionally, a **pilot VM or resource** in the pilot subscription to exercise a workload plan (e.g. a test VM for Defender for Servers, or a storage account for Defender for Storage). **Why:** A workload plan needs a resource to protect so the CWPP demonstration (e.g. EICAR on a server) has a target. **Verify:** `POCMDC-Enable.ps1 -WhatIf` lists candidate resources. **Implement-if-missing:** Deploy a small test VM (or use an existing test resource) in `MDC PoC – Pilot RG – Pilot`.

2.5 A Log Analytics workspace (where required)

Description: A **Log Analytics workspace** for continuous export / server assessment where the chosen plan uses one. **Why:** Some Defender-for-Cloud data and server assessments land in a workspace (per `global-requirements-defender.md` G7). **Verify:** A workspace exists and is access-controlled; `POCMDC-Enable.ps1 -WhatIf` reports it. **Implement-if-missing:** Provision (or reuse) a workspace scoped to the pilot subscription.

2.6 Evaluation criteria agreed up front

Description: A short, written list of what "pass" means — e.g. *the misconfiguration produced a recommendation, Secure Score changed, a security alert fired, and regulatory-compliance data populated.* **Why:** A PoC is only conclusive if the bar is set **before** you run it; the scorecard maps observations to these criteria. **Verify:** A criteria list exists; `POCMDC-Scorecard.ps1` accepts it as input. **Implement-if-missing:** Draft 5–8 criteria with the evaluating stakeholders before Step 1. *What counts as "good enough" is the customer's decision.*

2.7 Read access for validation

Description: Security Reader on the subscription (and `ThreatHunting.Read.All` where hunting is used). **Why:** The validate and scorecard scripts read recommendations, Secure Score, and alerts. **Verify:** `POCMDC-Validate.ps1` returns data; `Get-AzSecurityAlert` / `Get-AzSecuritySecureScore` succeed. **Implement-if-missing:** Grant Security Reader on the pilot subscription to the running account.

Checklist

- A pilot Azure subscription (or resource group `MDC PoC – Pilot RG – Pilot`) — `POCMDC-Enable.ps1 -WhatIf`
- Security Admin + Contributor to enable/tear down
- Awareness that Defender plans are billed while enabled — *verify current pricing*
- (Optional) a pilot VM/resource to exercise a workload plan
- A Log Analytics workspace where the plan requires one
- Evaluation criteria agreed and written down — `POCMDC-Scorecard.ps1`
- Security Reader (and hunting read) for validation

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud

Follow these steps **in order**. Work is in the **Azure portal** (`https://portal.azure.com` → **Microsoft Defender for Cloud**) with **Az PowerShell** helpers; alerts also surface in the unified Defender portal (`security.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model (run as a PoC):*** *Assess readiness, enable Defender for Cloud on **one pilot subscription**, generate **benign** posture + threat signal, validate the detections and score them, then **tear down** (including disabling the plan to stop billing). All test signal is sample alerts, a benign misconfiguration, and EICAR — **never a real attack or malware**. No other subscription is touched.*

Naming (set once, reuse verbatim)

- Pilot resource group: `MDC PoC – Pilot RG – Pilot`
- Benign misconfigured test resource: `MDC PoC Test Resource – Pilot RG – Pilot` (e.g. a storage account with public access)
- Scorecard output file: `MDC PoC Scorecard – Pilot Sub – Validate`
- Test-signal tag (Azure tag / notes): `MDC-PoC-TESTSIGNAL`

Step 1 — Assess readiness (read-only)

1. Sign in with `Connect-AzAccount` and select the pilot subscription.
2. Run `POCMDC-Enable.ps1 -WhatIf` — previews (changes nothing): your Azure roles, the pilot subscription, current Defender-for-Cloud plan state, candidate resources, and any workspace.
3. Create the pilot resource group `MDC PoC – Pilot RG – Pilot` (or confirm one to use). **Confirm the cost owner accepts the short Defender-plan billing window.**

Step 2 — Enable Defender for Cloud on the pilot subscription

1. `portal.azure.com` → **Microsoft Defender for Cloud** → **Environment settings** → **(pilot subscription)** → **Defender plans**.
2. Turn on **CSPM** (foundational is free; **Defender CSPM** is a paid tier — *verify current pricing/availability*) and **one workload plan** (e.g. **Defender for Servers** or **Defender for Storage**) for the pilot subscription only.
3. Alternatively run `POCMDC-Enable.ps1` (without `-WhatIf`) — it is `-WhatIf`-gated and enables CSPM + the chosen plan on the pilot subscription via `Az.Security`, showing the plan before it applies.
4. Confirm the plans show **On** for the pilot subscription.

Step 3 — Generate benign test signal

1. Run `POCMDC-TestSignal.ps1` (or follow its printed steps). It performs only **benign, documented** actions tagged `MDC-PoC-TESTSIGNAL`:

- **Sample security alerts:** trigger Defender for Cloud's built-in **sample alerts** — `portal.azure.com` → **Microsoft Defender for Cloud** → **Security alerts** → **Sample alerts** → select the pilot subscription

and create. *These are Microsoft-provided benign sample alerts for evaluation.*

• **Benign misconfiguration (CSPM):** deploy `MDC PoC Test Resource – Pilot RG – Pilot` — a deliberately-misconfigured **test** resource (e.g. a storage account with public blob access) so Defender for Cloud raises a **recommendation** and Secure Score reflects it. It holds **no real data**.

• **Workload threat (optional):** on a pilot VM covered by Defender for Servers, drop the **EICAR** test file to raise a benign Defender-for-Servers alert.

1. **Do not** attack real resources, deploy real malware, or expose real data. A benign test resource and sample alerts are sufficient.

2. Note the time of each signal for the scorecard latency comparison.

Step 4 — Validate and demonstrate the detections

1. Run `POCMDC-Validate.ps1` — it confirms (read-only): the **recommendation** from the misconfiguration, the **Secure Score**, the **security alerts** (sample + EICAR), and **regulatory-compliance** data.

2. In the portal, walk the demo: **Microsoft Defender for Cloud** → **Recommendations** (the misconfig), → **Secure Score**, → **Security alerts** (sample + EICAR), → **Regulatory compliance**. This is the headline-capability demo for stakeholders.

3. Confirm nothing outside the pilot subscription was affected.

Step 5 — Score against your criteria

1. Run `POCMDC-Scorecard.ps1 -CriteriaPath .\criteria.json -OutFile "MDC PoC Scorecard – Pilot Sub – Validate.json"`. It maps each observation (recommendation raised, Secure Score present, alert fired, compliance populated) to your pass/fail criteria and writes the scorecard.

2. Review the scorecard with stakeholders — the decision artefact.

Step 6 — Tear down (clean rollback + stop billing)

1. Run `POCMDC-Teardown.ps1 -WhatIf` first to preview, then for real.

2. Teardown: **delete** the test resource `MDC PoC Test Resource – Pilot RG – Pilot` (and optionally the pilot RG), **disable** the Defender workload plan (and Defender CSPM) on the pilot subscription **to stop billing**, and **resolve/dismiss** the `MDC-PoC-TESTSIGNAL` sample alerts.

3. Keep the scorecard as your record.

Step 7 — Decide: expand or stop

1. If the PoC passed and you choose to proceed, move to a **multi-subscription / multi-cloud onboarding + CSPM remediation** engagement — a separate, adjacent solution (see *Where to go next*).

2. If not proceeding, confirm the plan is **Off** and the test resource is gone with `POCMDC-Validate.ps1`.

Order of operations

`Assess (-WhatIf) → enable CSPM + one workload plan (pilot sub) → sample alerts + benign misconfig + optional EICAR → validate + demonstrate (recommendations, Secure Score, alerts, compliance) → score vs criteria → teardown (-WhatIf then real, disable plan) → decide expand or stop`

5. Testing Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud

Run these checks during the PoC (Steps 2–6 of the Implementation Guide). Each test has a **pass/fail gate**. All signal is **benign** (sample alerts + a benign misconfigured test resource + EICAR). Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Defender plan enabled on the pilot subscription

Procedure: Run `POCMDC-Enable.ps1 -WhatIf`, then confirm CSPM + the chosen workload plan show **On** for the pilot subscription only. **Pass:** The plans are on for the pilot subscription and no other. **Fail:** Plan off or wrong scope — see Troubleshooting *Plan not enabling*.

Test 2 — Benign signal generated

Procedure: Run `POCMDC-TestSignal.ps1`; confirm sample alerts were created, the benign test resource deployed, and (optionally) EICAR dropped on a pilot VM. **Pass:** All benign signals were generated; no real attack/malware used. **Fail:** A signal did not generate — see Troubleshooting *Signal not generated*.

Test 3 — Recommendation raised (CSPM)

Procedure: Microsoft Defender for Cloud → **Recommendations**; `POCMDC-Validate.ps1` confirms an assessment for the misconfigured `MDC PoC Test Resource – Pilot RG – Pilot`. **Pass:** A recommendation is raised for the benign misconfiguration. **Fail:** No recommendation — allow warm-up; see Troubleshooting *No recommendation*.

Test 4 — Secure Score reflects posture

Procedure: Microsoft Defender for Cloud → **Secure Score**; `POCMDC-Validate.ps1` returns the score. **Pass:** A Secure Score is present and reflects the pilot posture. **Fail:** No score — allow warm-up; see Troubleshooting *No Secure Score*.

Test 5 — Security alert fired

Procedure: Microsoft Defender for Cloud → **Security alerts**; `POCMDC-Validate.ps1` checks `Get-AzSecurityAlert` for the sample + EICAR alerts. **Pass:** The sample alert(s) and (if used) the EICAR alert appear. **Fail:** No alert — see Troubleshooting *No security alert*.

Test 6 — Regulatory-compliance data populated

Procedure: Microsoft Defender for Cloud → **Regulatory compliance**; confirm the default benchmark shows control status for the pilot subscription. **Pass:** Compliance data is visible for the pilot subscription. **Fail:** Empty — allow warm-up; see Troubleshooting *No compliance data*.

Test 7 — Scorecard produced

Procedure: Run `POCMDC-Scorecard.ps1` and confirm `MDC PoC Scorecard – Pilot Sub – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** Missing observations — re-run the relevant validate step first.

Test 8 — Clean teardown (and billing stopped)

Procedure: Run `POCMDC-Teardown.ps1 -WhatIf`, then for real; re-run `POCMDC-Validate.ps1`. **Pass:** The test resource is deleted, the Defender plan is **Off** (billing stopped), and sample alerts resolved. **Fail:** Residue or plan still on — see Troubleshooting *Teardown left residue*.

Exit gate

Tests 1–7 pass to call the PoC conclusive; Test 8 passes to close it out cleanly **and stop billing**. Keep the scorecard file as the decision record.

6. Troubleshooting Guide — Proof-of-Concept Kit: Microsoft Defender for Cloud

Common field issues, in **symptom** → **cause** → **fix** form. Run `POCMDC-Enable.ps1 -WhatIf` and `POCMDC-Validate.ps1` first — most causes surface there. All test signal is benign (sample alerts + a benign misconfigured test resource + EICAR).

Plan not enabling

Symptom: CSPM or the workload plan does not turn **On** for the pilot subscription. **Cause:** Insufficient role (needs Security Admin/Contributor/Owner), the wrong subscription is selected, or a policy blocks the change. **Fix:** Confirm your role on the pilot subscription (**Access control (IAM)**); select the correct subscription (`Set-AzContext`); re-run `POCMDC-Enable.ps1 -WhatIf` and check the reported gaps.

Signal not generated

Symptom: Sample alerts, the test resource, or EICAR did not generate. **Cause:** The sample-alert action targeted the wrong subscription, the test-resource deployment failed (naming/quota), or the pilot VM is not covered by Defender for Servers. **Fix:** Create sample alerts against the pilot subscription; confirm the test resource deployed in `MDC PoC – Pilot RG – Pilot`; confirm the VM is covered before using EICAR. Re-run `POCMDC-TestSignal.ps1`.

No recommendation

Symptom: Test 3 fails — the misconfiguration does not raise a recommendation. **Cause:** CSPM assessment has not run yet, or the resource is not actually misconfigured in an assessed way. **Fix:** Allow assessment warm-up (recommendations populate over time); confirm the test resource has the intended misconfiguration (e.g. public blob access); re-run `POCMDC-Validate.ps1`.

No Secure Score

Symptom: Test 4 fails — Secure Score returns nothing. **Cause:** Assessment is too recent, or the account lacks read permission. **Fix:** Allow warm-up; confirm **Security Reader** on the subscription; re-run `Get-AzSecuritySecureScore` via `POCMDC-Validate.ps1`.

No security alert

Symptom: Test 5 fails — no sample or EICAR alert appears. **Cause:** Sample alerts were created on a different subscription, the workload plan is off, or the alert is still propagating. **Fix:** Recreate sample alerts on the pilot subscription; confirm the workload plan is **On**; allow propagation time; check **Security alerts** and re-run `POCMDC-Validate.ps1`.

No compliance data

Symptom: Test 6 fails — Regulatory compliance shows nothing for the pilot subscription. **Cause:** The compliance assessment has not populated, or the default benchmark is not applied. **Fix:** Allow warm-up; confirm the Microsoft Cloud Security Benchmark is applied to the subscription; re-check **Regulatory compliance**. *Some standards are plan-gated — verify availability.*

Script cannot authenticate

Symptom: `POCMDC-*.ps1` fails on connect or returns access-denied. **Cause:** The Az modules are missing, you are not signed in, or the account lacks the role. **Fix:** Let the script install the modules (it checks on run); `Connect-AzAccount` and `Set-AzContext` to the pilot subscription; confirm **Security Reader** (read) / **Security Admin + Contributor** (enable/teardown). Re-run.

Teardown left residue

Symptom: After `POCMDC-Teardown.ps1`, the test resource remains or the plan is still **On** (still billing). **Cause:** Teardown ran in `-WhatIf` only, or a delete/disable failed on a permission. **Fix:** Run teardown **without** `-WhatIf`; confirm the plan shows **Off** in **Environment settings** → **Defender plans** (this stops billing); delete `MDC PoC Test Resource – Pilot RG – Pilot` (and the RG if PoC-only); resolve the `MDC-PoC-TESTSIGNAL` alerts; re-confirm with `POCMDC-Validate.ps1`.

7. Best Practices — Proof-of-Concept Kit: Microsoft Defender for Cloud

Field-proven guidance for running a clean, safe, cost-bounded Defender-for-Cloud proof of concept.

Keep the PoC scoped and reversible

• Enable Defender on **one pilot subscription** (or resource group) only. The value is that you can disable it cleanly with no impact to other subscriptions.

• Preview every change with `-WhatIf` first (`POCMDC-Enable.ps1`, `POCMDC-Teardown.ps1`).

Mind the cost — disable at teardown

• **Defender workload plans are billed while enabled.** Keep the PoC window short, enable only **one** workload plan, and **disable it at teardown** (`POCMDC-Teardown.ps1`). Diarise the teardown so a plan is never left on by accident. **Verify current pricing before enabling.**

Benign signal only — never real attacks

• Use **sample security alerts**, a **benign misconfigured test resource**, and **EICAR** exclusively. They exercise CSPM and CWPP without touching real workloads.

• **Never** attack a real resource, deploy real malware, or expose real data to "test." A benign test storage account with public access is enough to raise a recommendation; sample alerts are enough to prove the alert pipeline.

Use a disposable test resource for CSPM

• Deploy `MDC PoC Test Resource – Pilot RG – Pilot` specifically for the PoC, holding **no real data**, and delete it at teardown. This gives a clean, attributable recommendation and leaves nothing behind.

Set the bar before you run

• Agree the **evaluation criteria** in writing first (Requirement 2.6). Feed them into `POCMDC-Scorecard.ps1` so the verdict is objective.

Allow warm-up before judging

• Secure Score, recommendations, and regulatory-compliance data populate over **minutes to hours** after enabling. Give the pipeline time before scoring a "fail," and use `POCMDC-Validate.ps1` to check when signal has arrived.

Tear down completely and confirm

• Delete the test resource, disable the plan (confirm **Off** to stop billing), and resolve the `MDC-PoC-TESTSIGNAL` sample alerts. Then confirm with `POCMDC-Validate.ps1`. Keep only the scorecard.

Defer the decisions to the owners

• Whether the PoC "passed," which recommendations are "must-fix," and regulatory-mapping determinations are **the customer's** security/compliance decisions. This kit delivers the technical proof, not the verdict.

Where to go next

Once the PoC proves Defender for Cloud, natural next steps in the catalogue:

- The **Defender for Cloud — CSPM & Secure Score Remediation** solution to operationalise posture management across subscriptions.
- A **multi-subscription / multi-cloud (AWS/GCP) onboarding** engagement to extend coverage.
- The **Defender XDR Deployment & Unified Onboarding** solution to correlate Defender-for-Cloud alerts with Endpoint, Identity, Office, and Cloud Apps.
- The **Sentinel SOC/XDR integration** solution to stream Defender-for-Cloud into a SIEM for long-term hunting.

8. Deliverables Summary — Proof-of-Concept Kit: Microsoft Defender for Cloud

Documentation

Document	Purpose
Executive Summary	What the PoC proves, the boundary, cost note, and the effort
1. Solution Overview	What Defender for Cloud (CSPM + CWPP) is and how a scoped, safe PoC proves it
2. Requirements Guide	PoC-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable on a pilot subscription, generate benign signal, validate, score, tear down
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Safe, cost-bounded PoC guidance and where to go next

PowerShell scripts (Az PowerShell / Az.Security + Microsoft Graph + KQL)

Script	What it does
`POCMDC-Enable.ps1`	`-WhatIf`-gated pilot enable — previews readiness, then enables CSPM + one Defender workload plan on the pilot subscription (safe preview by default)

Script	What it does
`POCMDC-TestSignal.ps1`	**Benign** test-signal helper — triggers sample security alerts, deploys a benign misconfigured test resource, and guides an optional EICAR trigger on a pilot VM (no real attack/malware, ever)
`POCMDC-Validate.ps1`	Read-only demo/validate — confirms the recommendation, Secure Score, security alerts, and regulatory-compliance data
`POCMDC-Scorecard.ps1`	Scores observations against your evaluation criteria and writes `MDC PoC Scorecard – Pilot Sub – Validate`
`POCMDC-Teardown.ps1`	`-WhatIf`-gated clean rollback — deletes the test resource, **disables the Defender plan to stop billing**, resolves sample alerts

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — run here as a safe, reversible, cost-bounded PoC.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A concrete, in-environment proof that Microsoft Defender for Cloud delivers CSPM & workload protection — a recommendation and Secure Score from a benign misconfiguration, security alerts from sample alerts and EICAR, and regulatory-compliance visibility — scored against your criteria and cleanly torn down, with the Defender plan disabled to stop billing.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

POCMDC-Enable.ps1

Purpose: Enables Defender for Cloud on ONE pilot subscription only: foundational/Defender CSPM plus ONE workload plan (e.g. VirtualMachines / StorageAccounts). It previews readiness (roles, subscription, current plan state, candidate resources) and, gated behind -WhatIf, turns on the plans via Az.Security. NOTE: Defender workload plans are BILLED while enabled — keep the PoC short and disable at teardown. Verify current pricing before enabling. HOW TO RUN Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDC-Enable.ps1 -WhatIf Apply on the pilot subscription (enable CSPM + one workload plan): powershell -ExecutionPolicy Bypass -File .\POCMDC-Enable.ps1 -SubscriptionId <guid> -WorkloadPlan VirtualMachines Requires Az.Accounts + Az.Security and Security Admin/Contributor on the subscription. EXPECTED RESULTS A readiness summary, then (when applied) CSPM + the chosen workload plan set to On for the pilot subscription only. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped. Billed while on — reverse with POCMDC-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SubscriptionId, [ValidateSet('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults','AppServices')] [string]$WorkloadPlan = 'VirtualMachines', [string]$PilotResourceGroup = 'MDC PoC - Pilot RG - Pilot')

How to run: Preview only (changes nothing): powershell -ExecutionPolicy Bypass -File .\POCMDC-Enable.ps1 -WhatIf Apply on the pilot subscription (enable CSPM + one workload plan): powershell -ExecutionPolicy Bypass -File .\POCMDC-Enable.ps1 -SubscriptionId <guid> -WorkloadPlan VirtualMachines Requires Az.Accounts + Az.Security and Security Admin/Contributor on the subscription. EXPECTED RESULTS A readiness summary, then (when applied) CSPM + the chosen workload plan set to On for the pilot subscription only. .NOTES White-label. Read-only by default (-WhatIf). Pilot-scoped. Billed while on — reverse with POCMDC-Teardown.ps1. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SubscriptionId, [ValidateSet('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults','AppServices')] [string]$WorkloadPlan = 'VirtualMachines', [string]$PilotResourceGroup = 'MDC PoC - Pilot RG - Pilot')

POCMDC-Scorecard.ps1

Purpose: Turns the PoC observations into an objective, shareable scorecard. It re-checks the key Defender-for-Cloud capabilities (recommendations, Secure Score, security alerts) via read-only Az.Security, maps each to the pass/fail criteria you agreed up front, and writes a JSON + console scorecard. Confirm Regulatory compliance in the portal for that criterion. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDC-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDC PoC Scorecard - Pilot Sub - Validate.json" Sample criteria.json: [{ "id":"reco", "label":"Recommendation raised", "capability":"reco", "required":true }, { "id":"score", "label":"Secure Score present", "capability":"score", "required":true }, { "id":"alert", "label":"Security alert fired", "capability":"alert", "required":true }, { "id":"comp", "label":"Regulatory compliance data","capability":"portal","required":false }] Requires Az.Accounts + Az.Security and Security Reader. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDC PoC Scorecard - Pilot Sub - Validate.json', [string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDC-Scorecard.ps1 ` -CriteriaPath .\criteria.json -OutFile "MDC PoC Scorecard - Pilot Sub - Validate.json" Sample criteria.json: [{ "id":"reco", "label":"Recommendation raised", "capability":"reco", "required":true }, { "id":"score", "label":"Secure Score present", "capability":"score", "required":true }, { "id":"alert", "label":"Security alert fired", "capability":"alert", "required":true }, { "id":"comp", "label":"Regulatory compliance data","capability":"portal","required":false }] Requires Az.Accounts + Az.Security and Security Reader. EXPECTED RESULTS A per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([string]$CriteriaPath, [string]$OutFile = 'MDC PoC Scorecard - Pilot Sub - Validate.json', [string]$SubscriptionId)

POCMDC-Teardown.ps1

Purpose: Returns the pilot subscription to its pre-PoC state and stops Defender-plan billing. It: - deletes the benign misconfigured test resource(s) tagged MDC-PoC-TESTSIGNAL, - disables the chosen Defender workload plan (sets it back to Free) to STOP BILLING, - reminds you to dismiss the sample alerts. Every change is gated behind -WhatIf: run with -WhatIf first to preview, then without to apply. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDC-Teardown.ps1 -WhatIf Apply (delete test resource + disable plan): powershell -ExecutionPolicy Bypass -File .\POCMDC-Teardown.ps1 -SubscriptionId <guid> -WorkloadPlan VirtualMachines -ResourceGroup "MDC PoC - Pilot RG - Pilot" Requires Az.Accounts + Az.Security (+ Az.Resources) and Security Admin/Contributor. EXPECTED RESULTS The test resource(s) deleted, the Defender plan set back to Free (billing stopped), and a reminder to dismiss the sample alerts. .NOTES White-label. Reversible teardown. Disabling the plan is what stops the Defender-for-Cloud billing. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SubscriptionId, [ValidateSet('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults','AppServices')] [string]$WorkloadPlan = 'VirtualMachines', [string]$ResourceGroup = 'MDC PoC - Pilot RG - Pilot', [string]$Tag = 'MDC-PoC-TESTSIGNAL', [switch]$DeleteResourceGroup)

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDC-Teardown.ps1 -WhatIf Apply (delete test resource + disable plan): powershell -ExecutionPolicy Bypass -File .\POCMDC-Teardown.ps1 -SubscriptionId <guid> -WorkloadPlan VirtualMachines -ResourceGroup "MDC PoC - Pilot RG - Pilot" Requires Az.Accounts + Az.Security (+ Az.Resources) and Security Admin/Contributor. EXPECTED RESULTS The test resource(s) deleted, the Defender plan set back to Free (billing stopped), and a reminder to dismiss the sample alerts. .NOTES White-label. Reversible teardown. Disabling the plan is what stops the Defender-for-Cloud billing. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([string]$SubscriptionId, [ValidateSet('VirtualMachines','StorageAccounts','SqlServers','Containers','KeyVaults','AppServices')] [string]$WorkloadPlan = 'VirtualMachines', [string]$ResourceGroup = 'MDC PoC - Pilot RG - Pilot', [string]$Tag = 'MDC-PoC-TESTSIGNAL', [switch]$DeleteResourceGroup)

POCMDC-TestSignal.ps1

Purpose: Produces harmless posture + threat signal so Defender for Cloud has something to surface. It: 1. Guides creation of Defender for Cloud's built-in SAMPLE SECURITY ALERTS (portal), a Microsoft-provided benign way to populate the alert pipeline. 2. Optionally deploys a benign, DELIBERATELY-MISCONFIGURED test storage account (public blob access) in the pilot resource group so CSPM raises a recommendation. It holds NO real data and is deleted at teardown. 3. Prints the steps to drop the EICAR test file on a pilot VM (Defender for Servers) for a benign workload alert. NO real attack,

real malware, or real data exposure is ever performed. -WhatIf previews. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\POCMDC-TestSignal.ps1 -WhatIf Deploy the benign misconfig test resource: powershell -ExecutionPolicy Bypass -File .\POCMDC-TestSignal.ps1 -ResourceGroup "MDC PoC - Pilot RG - Pilot" -Location eastus -DeployTestResource Requires Az.Accounts + Az.Storage and Contributor on the pilot subscription. EXPECTED RESULTS Sample alerts populate, a recommendation is raised for the benign misconfig, and (optionally) an EICAR alert fires — for the PoC to validate. .NOTES White-label. BENIGN signal only (sample alerts + benign misconfig test resource + EICAR). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$ResourceGroup = 'MDC PoC - Pilot RG - Pilot', [string]$Location = 'eastus', [switch]$DeployTestResource, [string]$Tag = 'MDC-PoC-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\POCMDC-TestSignal.ps1 -WhatIf Deploy the benign misconfig test resource: powershell -ExecutionPolicy Bypass -File .\POCMDC-TestSignal.ps1 -ResourceGroup "MDC PoC - Pilot RG - Pilot" -Location eastus -DeployTestResource Requires Az.Accounts + Az.Storage and Contributor on the pilot subscription. EXPECTED RESULTS Sample alerts populate, a recommendation is raised for the benign misconfig, and (optionally) an EICAR alert fires — for the PoC to validate. .NOTES White-label. BENIGN signal only (sample alerts + benign misconfig test resource + EICAR). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$ResourceGroup = 'MDC PoC - Pilot RG - Pilot', [string]$Location = 'eastus', [switch]$DeployTestResource, [string]$Tag = 'MDC-PoC-TESTSIGNAL')

POCMDC-Validate.ps1

Purpose: Read-only validation and live-demo helper for the Defender for Cloud proof of concept. Using Az.Security, it confirms the benign test signal surfaced as: - recommendations (assessments) — CSPM saw the benign misconfiguration - Secure Score — posture score present - security alerts — sample + EICAR alerts fired It also points to the portal Regulatory compliance view for the demo. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\POCMDC-Validate.ps1 -SubscriptionId <guid> Requires Az.Accounts + Az.Security and Security Reader on the subscription. EXPECTED RESULTS A pass/fail line per capability (recommendations, Secure Score, alerts), plus the portal path for the regulatory-compliance demo. .NOTES White-label. Read-only. Az.Security reads posture/alerts only; it changes nothing. > [CmdletBinding()] param([string]$SubscriptionId)

How to run: powershell -ExecutionPolicy Bypass -File .\POCMDC-Validate.ps1 -SubscriptionId <guid> Requires Az.Accounts + Az.Security and Security Reader on the subscription. EXPECTED RESULTS A pass/fail line per capability (recommendations, Secure Score, alerts), plus the portal path for the regulatory-compliance demo. .NOTES White-label. Read-only. Az.Security reads posture/alerts only; it changes nothing. > [CmdletBinding()] param([string]$SubscriptionId)